

Introdução à Ciência da Computação com Python II

Busca, ordenação e desempenho

REALIZAR APÓS Módulo 5 / Semana 5

Módulos 4 a 5 | 120 minutos | 100 pontos

Curso principal: Introdução à Ciência da Computação com Python Parte 2 |
<https://www.coursera.org/learn/ciencia-computacao-python-conceitos-2>

Conteúdo verificável: Busca sequencial; seleção; complexidade; bolha; comparação; testes; busca binária

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Nota de fonte: Módulo 7 é explicitamente extra (Scrapy e PyGame); excluído das provas obrigatórias.

Busca e pré-condição

(a) Implemente busca sequencial retornando índice ou -1 . (b) Compare seu custo com busca binária. (c) Explique por que busca binária em $[3, 1, 4, 2]$ não tem garantia de correção.

Desenvolvimento / justificativa / esquema:

Seleção direta

(a) Execute duas passadas de seleção direta em [4, 1, 3, 2]. (b) Implemente a ordenação. (c) Dê número de comparações no pior caso e explique se depende da ordem inicial.

Desenvolvimento / justificativa / esquema:

Bolha melhorada

(a) Implemente bolha com parada se não houver trocas. (b) Trace[1, 2, 3] e[3, 2, 1]. (c) Proponha testes automatizados que verifiquem conteúdo e ordenação, não apenas comprimento.

Desenvolvimento / justificativa / esquema:

Busca binária e desempenho

Implemente busca binária iterativa em lista ordenada crescente. (a) Especifique comportamento com repetidos. (b) Teste vazio e extremos. (c) Planeje comparação experimental justa com sequencial, sem misturar custo de ordenação e busca.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Busca e pré-condição (15 pontos)

```
def busca(a, alvo):
    for i in range(len(a)):
        if a[i] == alvo: return i
    return -1
```

Sequencial pior caso $O(n)$, não requer ordenação. Binária $O(\log n)$ comparações em sequência ordenada com acesso apropriado. Sem ordenação, descartar metade pelo valor central pode excluir o alvo. O custo de ordenar antes deve entrar na decisão para uma busca isolada.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Seleção direta (25 pontos)

Passadas: [1, 4, 3, 2] e [1, 2, 3, 4].

```
def selecao(a):
    for i in range(len(a)-1):
        m = i
        for j in range(i+1, len(a)):
            if a[j] < a[m]: m = j
        a[i], a[m] = a[m], a[i]
```

Comparações $(n-1)+(n-2)+\dots+1=n(n-1)/2$, independente da ordem inicial nesta versão. Trocas e comparações são medidas distintas.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Bolha melhorada (25 pontos)

```
def bolha(a):
    for fim in range(len(a)-1, 0, -1):
        mudou = False
        for j in range(fim):
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
                mudou = True
        if not mudou: break
```

Ordenada termina após uma passagem; inversa requer todas. Testes: vazia, unitária, repetidos, negativos; verificar resultado igual à lista ordenada de referência e multiplicidades preservadas.

Pior $O(n^2)$, melhor $O(n)$.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Busca binária e desempenho (35 pontos)

```
def binaria(a, x):  
    lo, hi = 0, len(a)-1  
    while lo <= hi:  
        mid = (lo+hi)//2  
        if a[mid] == x: return mid  
        if a[mid] < x: lo = mid+1  
        else: hi = mid-1  
    return -1
```

Retorna qualquer ocorrência. Testar[], [1], primeiro/último, ausente e repetidos. Comparar mesmos alvos e listas, repetir medições, separar preparação e considerar sucesso/fracasso. Contar comparações ajuda a não confundir ruído de tempo com custo teórico.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.